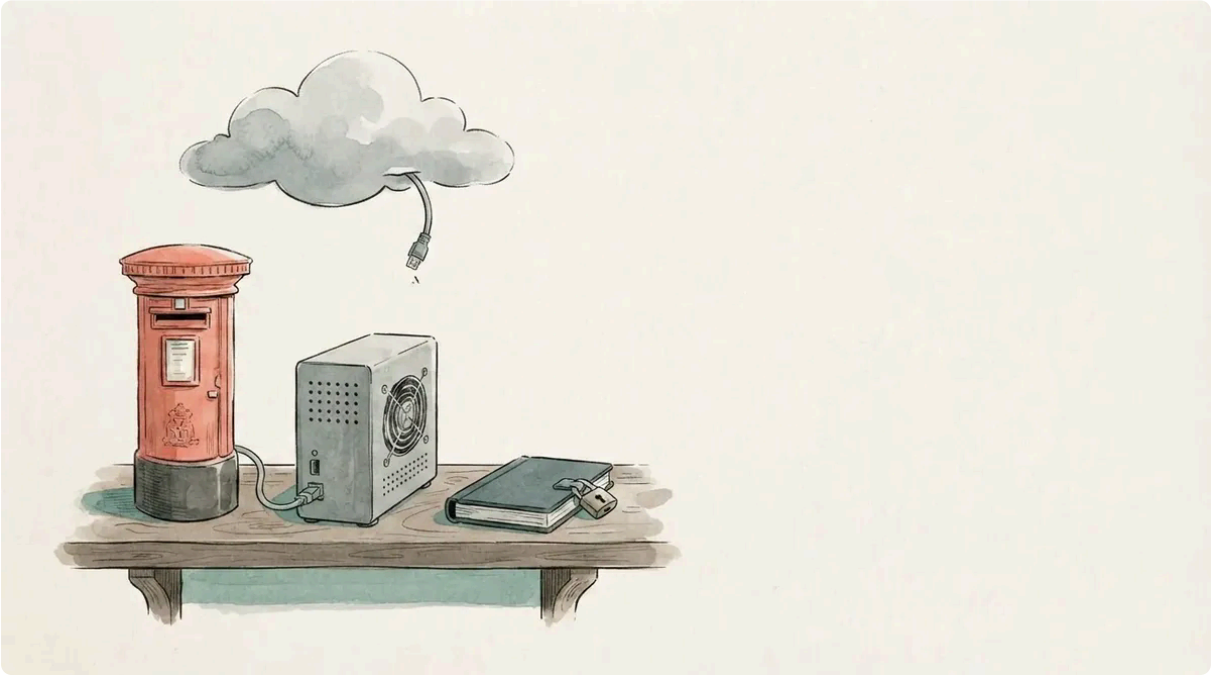
# A subscribe button does not need a Worker

Emil Lindfors | September 08, 2026



The newsletter dashboard had been behind a login for about an hour when it showed me this:

```
Some of this could not be read:
    subscriber list: JMAP answered 401
    event log: refused the list credential
    send log: refused the list credential
```

Three errors, one cause. The password the dashboard used to read the subscriber list had drifted from the one the Cloudflare Worker used to write it, and there was no way to check which was right, because a Worker secret cannot be read back. The list lived in a mailing-list object on my mail server. The history lived in WebDAV filenames. The lock that stopped an issue going out twice was an HTTP header. All three needed the same password, in two places, and the two places disagreed.

The Workers added needless complexity to what is only an API call to subscribe to the newsletter.

So the newsletter moved. By the end of the day the Worker was deleted, and the whole thing was one Rust service on the mail server with a Postgres behind it. This post is the

build log, the schema, and then a detour into what the research says about writing code like this with a coding agent, because I did, and the result is simpler than what I had.

## What February got right, and what it turned into

The February post made a virtue of having no database. Subscribe was a Worker on Cloudflare, the list was a Stalwart mailing list, and that was the whole system. With zero subscribers and a free tier, that was the right call. It is also how a mail server ended up being used as a key-value store.

At first I wrote to a mail store, which is quite unconventional but was a workaround as I didn't want to tag on more Cloudflare services.

Every feature after that had to find somewhere to keep a fact, and the answer was always another place on the mail server. Double opt-in kept its pending state inside a signed link, and that part was good. The send lock became a `PUT` with `If-None-Match: *` to a WebDAV folder. The event log became one JSON file per subscribe, confirm and unsubscribe, with the address hashed into the filename so the folder sorted by time. One file per fact, on a server whose job is mail. It worked. The Worker was 3,096 lines by the end.

Then I wanted better instrumentation: which issue went to whom, so I could send a series to the people who joined after it. That is a table. A Worker on Cloudflare cannot reach a Postgres on my server without one of three things: exposing the Postgres to the internet, paying for a tunnel, or adding a fourth Cloudflare product. And the dashboard that reads all this had moved to the server the day before, behind Kanidm. A public page holding the list credential was the one exposure that mattered.

Now as I moved the admin interface onto the server as well it was better to also colocate the API subscribe call so I could then just run all the send and other calls internally from the box.

Here is the before and after:

| | February | Now |
|---|---|---|
| Subscribe, confirm, unsubscribe | Worker at `lindfors.no/api/*`, WASM | `newsletter.lindfors.no`, three routes, nginx to a loopback port |
| The list | Stalwart mailing list, over JMAP across the internet | Postgres table, over loopback |
| Who got what | nowhere | `deliveries` table |
| The send lock | `If-None-Match: *` on WebDAV | a primary key |
| The event log | one JSON file per event | a table |

| | February | Now |
|---|---|---|
| Sending | JMAP to Stalwart, across the internet, 45 recipients maximum | JMAP to Stalwart on loopback, no cap |
| The dashboard | a separate service | the same binary, other routes |
| Credentials outside the box | list password in a Worker secret | none |

The 45-recipient cap deserves a line. Every message from a Worker is a subrequest, and Workers cap those at 50 per invocation on the free plan. The old code refused sends above 45 instead of truncating them. The cap is gone.

## The day, in order

Nothing here is hard. It is a list, and if you are about to do the same you will want it written down.

1. **Kanidm.** A public OAuth2 client for the dashboard, PKCE required, one group allowed in. The service reads the client's discovery document instead of hard-coding paths, so it moved from Keycloak to Kanidm without the browser flow changing.
2. **Postgres.** A role, a database, a `pg_hba` line for that pair, and a daily `pg_dump` into a root-only directory. There had been no database backup on the box at all, because there had been nothing in it worth backing up.
3. **The service.** The Worker's validation, signed links, mail templates and tests carried over almost unchanged. The JMAP list code and the WebDAV code did not carry over at all. The dashboard's code merged in. 43 tests, a 7 MB static binary, cross-compiled from Windows with no C toolchain because the binary contains no C.
4. **Two nginx names.** `newsletter.lindfors.no` behind the Cloudflare proxy, routing exactly three paths and answering 404 to everything else. `admin.lindfors.no` unproxied, behind the login, routing the rest. Both to the same port.
5. **The site.** The form's action changed to the new name, the CSP grew by one host, and a `_redirects` file sends the old `lindfors.no/api/*` links on with a 308, because the unsubscribe links in delivered mail never expire and a 301 would turn a one-click POST into a GET.
6. **Migration.** Three subscribers. One insert.
7. **The Worker.** Deleted.

Three things went wrong. None of them was interesting for long. The service account's group did not exist, because Alpine's `adduser -S` does not create one. The environment file executed half a secret, because the secret contained a semicolon and the init script sources the file with a shell. Every value in that file is single-quoted now. And the first

cutover failed with a 401 from Stalwart, because I had put the sender's password in the wrong file. I spent longer on that one than on the other two together, until the file's modification time showed my edit had never landed. The second cutover sent the confirmation mail, I clicked it, the welcome mail arrived with the recent posts listed, and that was the newsletter running on its own server.

## Four tables, one of them a lock

This is the schema. It is small enough to read in full, and if you copy one thing from this post, copy the lock.

```
CREATE TABLE subscribers (
    subject       text        PRIMARY KEY,   -- HMAC of the address, 16 hex
chars
    email_enc     bytea       NOT NULL,      -- the address, sealed
    subscribed_at timestamptz NOT NULL DEFAULT now(),
    source        text        NOT NULL DEFAULT 'confirmed'
);

CREATE TABLE events (
    id      bigint      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    at      timestamptz NOT NULL DEFAULT now(),
    kind    text        NOT NULL CHECK (kind IN ('requested', 'confirmed',
'unsubscribed')),
    subject text        NOT NULL
);

CREATE TABLE sends (
    slug        text        PRIMARY KEY,     -- the lock
    claimed_at  timestamptz NOT NULL DEFAULT now(),
    finished_at timestamptz,
    status      text        NOT NULL DEFAULT 'sending',
    recipients  integer     NOT NULL,
    sent        integer     NOT NULL DEFAULT 0,
    failed      text[]      NOT NULL DEFAULT '{}'
);

CREATE TABLE deliveries (
    slug      text  NOT NULL REFERENCES sends (slug) ON DELETE CASCADE,
    subject   text  NOT NULL,
    email_enc bytea NOT NULL,
    at        timestamptz NOT NULL DEFAULT now(),
    status    text  NOT NULL CHECK (status IN ('sent', 'failed', 'assumed')),
    PRIMARY KEY (slug, subject)
);
```

Three things to note here.

**The send lock is the primary key on `sends`.** A send starts with an insert of the slug. If the row exists, the insert fails, and no message goes out. That is the whole idempotency guard. It replaced a paragraph of reasoning about WebDAV preconditions in the old code. A partial send leaves the row with `status = 'partial'` and the missed addresses in `failed`.

**No address is stored in the clear.** Every address is sealed with XChaCha20-Poly1305 under a key that exists only in the service's environment file, and every row is keyed by an HMAC of the address instead. A lookup, an insert or an unsubscribe never decrypts anything. A copy of the database names nobody, and the nightly dump is safe to keep. The service decrypts in exactly two places: when it is about to send, and when the dashboard shows me who got what. Lose the key and the list is gone, so the key is in my password manager as well. That is the one operational rule this design adds.

**`deliveries` is what the whole move was for.** One row per recipient per issue, written as each message is accepted or refused. With it, a send has a second mode:

```
$ site-tools newsletter send how-i-built-this-site --catch-up
Catch-up: only subscribers who have not received this issue.
Send to the subscribers who have not had it? [y/N] y
{"success":true,"sent":1,"skipped":3}
```

A catch-up mails an issue only to subscribers with no delivery row for it. So a series can go to whoever joined after it, one post at a time, and nobody gets anything twice. The two issues sent under the old system were marked as delivered to the three people who were on the list then, with a status of `assumed`, so the record starts honest.

The dashboard reads the same tables. It has a *Missing* column per issue, which is the number of current subscribers without that issue, and a subscribers table with what each has had.

So now I have much better instrumentation and control over the subscribers and the metrics of the visitors to my site on my own server, safe and sound.

## What listens where

The thing I like most about the result is the list of what you can reach from the internet. It got shorter.

| Listens | Reachable from | Holds |
|---|---|---|
| `newsletter.lindfors.no` | anyone, through the Cloudflare proxy, rate limited twice | three routes, no credential |
| `admin.lindfors.no` | anyone, to a login page | nothing until Kanidm says yes |
| the service, `127.0.0.1:8788` | nginx on the box | the sealed list |

| Listens | Reachable from | Holds |
|---|---|---|
| Postgres, `127.0.0.1:5432` | processes on the box | the tables |
| Stalwart JMAP, `127.0.0.1:8080` | processes on the box | the sender's app password, presented by the service |

Every upstream is a port on the same machine, in plain HTTP. That is what lets the binary carry no TLS stack. Where something only speaks TLS, Kanidm and Cloudflare Pages, nginx holds a plain listener on loopback in front of it. The service refuses an `https://` URL at startup and names the variable. Without that check, reqwest would reject the URL on the first request, far from the line that caused it.

Rate limits are in two layers: nginx keys on the address Cloudflare reports, and the process keys on the address and on the typed email, which nginx cannot see. The old Worker had Cloudflare's rate-limiting bindings. This has a map in a mutex and the same numbers.

What did not change: the confirmation link is still `HMAC(secret, "confirm:v1:<exp>:<email>")` and still lives in the link, not in a table. There is a database now, and you still do not want a row for something that lives for two days.

## Writing this with a coding agent, and the research on what goes wrong

Now the part that is about method. I did not write most of this code. A coding agent did, over one working day, from a design I gave it and pushed on while it worked. That is the setup every study of LLM-written code warns about, so read what the studies found before you decide what to make of the day.

The findings, in plain terms:

- Pearce et al. (2022) gave GitHub Copilot 89 security-relevant scenarios and found about 40% of its 1,689 suggested programs vulnerable. The model reproduces the common way to write a thing, and the common way is often the insecure way.
- Perry et al. (2023) ran a user study: people with an AI assistant wrote less secure code than people without one, and were more likely to believe their code was secure. Both halves of that matter.
- Liu et al. (2024) characterised the quality problems in ChatGPT's code across two thousand tasks, from wrong output to maintainability smells, and showed that the same model fixes a good share of them when it is told precisely what is wrong.
- Dakhel et al. (2023) compared Copilot with human programmers and found its solutions less often correct, and its wrong solutions cheaper to repair than a human's wrong solutions.

Read together, these say three things. The model writes the default. The default is often wrong in ways the person prompting cannot see. The fix is feedback that names the defect. None of them is about architecture, and that is the gap the day fills in, as one data point.

I'm simplifying and making things better as opposed to using LLMs for vibe coding where the complexity often creeps and bad choices are made all the time.

The difference between this and vibe coding is who holds the shape. The shape was decided before any code:

- one binary, one port, two nginx names
- no TLS in the binary, every upstream on loopback
- the lock as a primary key
- addresses sealed, rows keyed by a pseudonym
- fail closed on the send, fail open on the audit log
- the public name routes three paths and nothing else

Those are architecture decisions, and each one is a constraint the agent could not talk its way around. Within them, the agent did the mechanics: the axum handlers, the migration that seals the rows in a transaction, the OpenRC script, the nginx snippets, the tests, the docs. When it hit the semicolon in the secret, it fixed the script that assembles the file so every value is quoted. Fable 5.1 was excellent at getting this right from the start, and with some guidance produced an architecture I am satisfied with.

Three things I would tell you if you are about to do the same:

- **Decide the constraints first and write them where the agent reads them.** Mine live in a `CLAUDE.md` at the repo root and in the README of each service. "No TLS in the binary" is one line and it decided the whole deployment story.
- **Make the agent prove things instead of describing them.** Every step above ended in a curl, a test run or a query against the live tables. The smoke test that ran the new binary on a side port before it replaced the old one is what caught the semicolon. Perry's finding about confidence applies to you when you read the agent's summary, too.
- **Refuse complexity out loud.** Say no to the extra listener and the second rate limiter, and say why, so the constraint ends up in the docs the agent reads next time. Complexity creep in agent-written code is not the agent's ambition. It is nobody saying no.

The tell is that the codebase got smaller. 3,096 lines of Worker plus about 1,100 of a separate dashboard became 3,457 lines of one service, with more tests, more features, and no credential outside the box.

## What comes next

The weekly send is half built. A job on the box now publishes one queued post a week and mails its issue in the same run, and this post is the first to go out that way. The other

half, sending the old issue the most subscribers are missing when there is nothing new, is designed and not built. The `deliveries` table is what makes it possible.

Bounce handling is still the open item it was in February. An address that hard-bounces stays on the list, and the send report has the raw material for the rule I have not written.

The code is on GitHub under `newsletter/`, with a README that argues the case at more length than this. If you run a static site with a Worker-shaped newsletter and a VPS with nothing on it, the move is one day, and the day is mostly nginx.